INTENT BEFORE ACTION: DECODABLE AND PORTABLE SUB-GOAL REPRESENTATIONS IN LLMS

Anonymous authors
Paper under double-blind review

ABSTRACT

When an LLM writes code to satisfy a multi-step specification, does it know *what it intends to do next* before it starts writing? We show that code LLMs encode a compact representation of their next instrumental sub-goal before emitting its implementation, and that this encoding is *decodable* and *portable*. Using causal mediation analysis, we localize this encoding to a set of attention heads, which we call the *goal heads*. We introduce GOALSCOPE, a lightweight probe that uses what these heads write to the residual stream to verbalize the upcoming sub-task in natural language. We demonstrate that the encoding is abstract: it can be extracted from one context and transplanted to trigger the same sub-goal in a different context and programming language. These heads also mediate the model's own implementation: preserving their access to task information largely preserves the capability of executing the next sub-task, while restricting that access in a broader set of goal heads sharply impairs it. We replicate the readout and transfer findings across model families and sizes, and explore the same heads' readouts in non-code settings. Our results provide concrete, testable evidence that LLMs encode abstract representations of intermediate goals before realizing them in tokens.

1 INTRODUCTION

arnab: I am using terms like "sub-goal", "intention", ... interchangeably in the intro. We need to come to a consensus on what term to use.

Consider a programmer asked to implement *merge sort*, a task that decomposes into several sub-tasks: *split the array*, *sort the two halves*, and *merge the sorted halves*. Before writing the loop that combines the two sorted arrays, the programmer already has a coherent goal: *"merge the two halves."* We ask whether large language models (LLMs) write code the same way: when an LLM emits a long span of tokens to satisfy a multi-step specification, does it hold an explicit representation of the *next* instrumental sub-goal before it begins its implementation?

We study the *instrumental* level of this hierarchy: the *broader goal* (implement merge sort) decomposes into *instrumental sub-goals* (merge the two sorted arrays), each of which admits many *implementations* (such as using a *for* loop or a *while* loop). Our question concerns a representation of *which sub-goal the model is about to pursue*, and whether that representation can be separated from its concrete realizations in code.

Although LLMs are trained to predict one token at a time, a growing body of work shows that they *look ahead*: they encode information about tokens they have not yet emitted (Pal et al., 2023) and plan toward constraints several tokens in the future (Lindsey et al., 2025). We ask what the plan for the *next step* looks like as a concrete internal object, and whether that object is one we can read and move between contexts.

arnab: maybe move to Related Works if the intro gets long.

We investigate this question through activation patching on code tasks with multiple sub-goals. Our main experiments use `Qwen2.5-Coder-32B-Instruct`, with replications across model families and sizes. Our analysis yields three key findings.

The next sub-goal is encoded before implementation, and is verbalizable (Section 3). We localize the encoding of the LM's next sub-goal to a compact set of attention heads, which we call the *goal heads*. We introduce GOALSCOPE, a lightweight probe built on these heads that verbalizes the upcoming sub-goal directly from the LM's internal activations. Before the model writes the

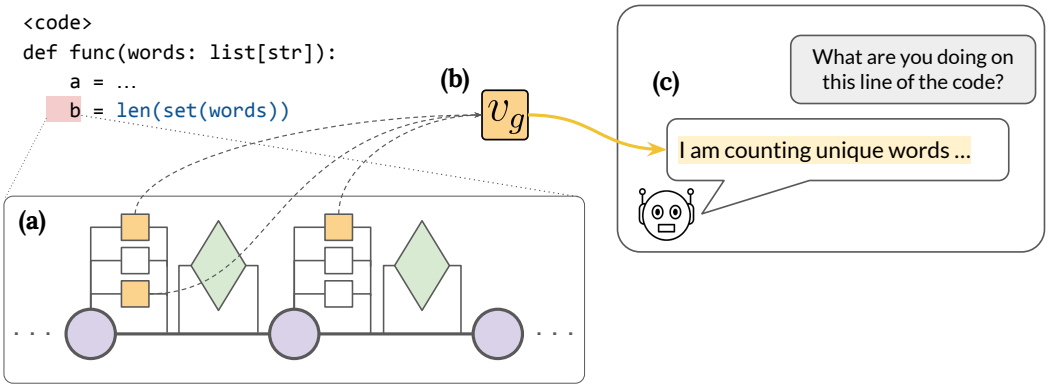


Figure 1: **The GoalScope intervention.** (a) At the highlighted pivot (b), we cache the selected attention heads' outputs that are added to the residual stream. (b) We aggregate these outputs into a single vector v_g. (c) In a separate LM run, the user asks "What are you doing on this line of the code?" We steer the response by adding v_g at each generation position. The query can be adapted to elicit different aspects of the model's encoding of its context and upcoming behavior. Blue source code follows the pivot and is shown only for context; it is unavailable to the cached state.

implementation, we can extract what the heads write to the residual stream and use it to generate a faithful description of the operation it is about to perform.

The encoding is abstract and portable (Section 3.2). Transplanting the encoding steers a fresh code context to pursue the source sub-goal. A goal extracted from Python can trigger the same operation in C++, and vice versa. This portability provides evidence that the heads carry a representation of the operation that generalizes across its surface realizations. We also explore whether the same heads can verbalize sub-goals in code whose component steps are not explicitly listed, and inferred relations in non-code few-shot tasks (Section 6).

Goal heads mediate the model's implementation of the sub-goal (Sections 4 and 5). A small set of heads suffices for effective readout, and a broader set is causally important for carrying out the operation. When we restrict other upper-layer heads' access to the upcoming task and code span, preserving goal-head access largely preserves implementation performance. Conversely, restricting the broader goal-head set to the attention sink sharply reduces performance, while restricting random heads has little effect. The heads attend to relevant task descriptions, and changing their query or key/value states can redirect the sub-goal pursued. Using their value and output projections, we also introduce **OVReader** to trace where the upcoming sub-goal becomes readable in intermediate residual states.

Our results provide concrete, testable evidence that code LLMs represent and pursue intermediate goals: a compact, portable, causally manipulable encoding of *intent* that precedes action.

arnab: Teaser figure: the readout + one steering example, framed as "intent before action".

2 Task and Intervention Setup

2.1 Language models and head contributions

An autoregressive transformer $M : \mathcal{X} \to \mathcal{Y}$, over a vocabulary $\mathcal{V}$ maps a sequence of tokens $x = (x_1, \ldots, x_n)$, $x \in \mathcal{V}$, to $y \in \mathbb{R}^{|\mathcal{V}|}$, a probability distribution over its next token. Internally, M has L layers; the residual stream at layer ℓ and position t is $h_t^\ell = h_t^{\ell-1} + m_t^\ell + \sum_j a_t^{\ell j}$, where m_t^ℓ is the output of the MLP and $a_t^{\ell j}$ is the contribution of the j-th attention head. Suppressing

normalization and positional encoding, a head's contribution to the residual stream is

$$a_t^{\ell j} = \left(\sum_{i \le t} \mathrm{Attn}(q_t, K)_i v_i \right) W_O^{\ell j}, \qquad \mathrm{Attn}(q_t, K) = \mathrm{softmax}\left(\frac{q_t K^\top}{\sqrt{d_{\mathrm{head}}}} \right). \tag{1}$$

with query $q_t = h_t^{\ell-1} W_Q^{\ell j}$, keys $K = h_{\le t}^{\ell-1} W_K^{\ell j}$, affinity of query and key states determine the mixture over the value states $v_i = h_i^{\ell-1} W_V^{\ell j}$, which is then projected with the output projection $W_O^{\ell j}$ and added to the residual stream. In grouped-query attention, multiple query heads read from the same key/value head.

2.2 Sequential subgoal implementation

We study a set of controlled settings in which the model must satisfy a *broader goal* that decomposes into an ordered set of independent *sub-tasks* $G = \{g_1, \ldots, g_K\}$, each specified in natural language (for example, *"check if there are any palindromes in a list of strings"*). A prompt p presents the specification and asks the model to implement the sub-tasks in code, one after another. The model realizes each g_k over a contiguous span of generated tokens. We use two terms throughout. The *pivot* π_k is the token position at which the model is about to begin sub-task g_k, which we dub as the *frontier*. In our controlled setup, we prefill in the LM's progress until the *pivot* π_k. This let's us ask if the upcoming assignment is already encoded in the *pivot* π_k and compare the readout with the LM's actual *future* behavior. A concrete example of the task is shown on Figure 1, with more details on Appendix A.

Our main evaluation contains 253 specifications over integers, strings, lists of integers, and lists of strings. Each prompt lists two to four operations, which we refer to as predicates, including Boolean tests, counts, and other transformations. We inspect five positions around the frontier: the end of the preceding line, indentation, the variable name, the assignment operator, and the first implementation token. The first four are pre-implementation capture positions, which we call *pivots*; the fifth provides a comparison after implementation has begun. We use the assignment operator as the principal pivot for the ablation experiments. The corpus supplies 1,265 observations per evaluation mode, with five repeated measurements per specification. The data distribution, complete example prompt, and tokenization are in Appendices A and C.1.

arnab: condense the different pivot explanation to figure. nothing in text. stash specifics in the appendix.

2.3 Intervention Primitives

Activation patching (Vig et al., 2020; Mueller et al., 2026). We run M on a prompt p but overwrite a chosen set of internal activations with latents cached from a different run, and read out the effect on the output distribution. We write $M(p)\left[a_i^{\ell j} \leftarrow v\right]$ for the forward pass on p in which the output of head $[\ell, j]$ at position i is replaced by the vector v. When the replaced value is cached from a *source* prompt p_{src} and injected into a *destination* prompt p_{dst}, a change in $M(p)\left[v_{\mathrm{orig}} \leftarrow v_{\mathrm{rep}}\right]$ isolates the causal role of the patched component/encoding.

arnab: M(x) usually denotes a single forward pass. Our setup is multi-token generation. Ok?

Query, key/value, and output transplants. The head decomposition in Eq. 1 exposes three places a transplant can act, and we use all three. An *output* (OV) transplant replaces a head's contribution $a_t^{\ell j}$ (what it *writes* to the residual stream), and is the primitive behind GoalScope and our head localization (Sections 3 and 4). A *query* transplant replaces the query $q_t^{\ell j}$ that the destination uses at π_k with the source's, changing what the read *looks for*. A *key/value* transplant replaces the keys and values $K^{\ell j}, V^{\ell j}$ at the specification tokens with the source's, changing what the read *finds*. We use these transplants both to *steer* a fresh context toward an intended sub-goal and to probe *where* the goal is computed (Section 5).

3 The Next Sub-Goal Is Encoded Before Implementation

Can we read what the model is about to do before it writes the implementation? We first locate attention heads that carry a portable representation of a code operation. We then use their contributions at the pivot to describe the upcoming sub-goal and to trigger its implementation in a new context.

3.1 LOCALIZING GOAL HEADS

We begin by locating a set of attention heads (*goal heads*) that carry the encoding of the next sub-goal on a simpler, single-predicate task. A source and a destination prompt request different predicates on the same datatype, written in different programming languages. Both prompts end where the next token distinguishes the requested implementations. We cache the source head contributions and learn a sparse mask $m^{\ell j}$ jointly over attention heads. In the destination run, the mask controls which contributions are replaced:

$$a^{\ell j}_{-1} \leftarrow m^{\ell j} a^{\ell j}_{\text{src}} + (1 - m^{\ell j}) a^{\ell j}_{-1}. \tag{2}$$

The mask is optimized to promote a token that would begin implementing the *source predicate in the destination language*. We choose this target to differ from the source's next token, so the localization objective rewards transfer across different token realizations. We call the selected set H the *goal heads*. Our main selection contains 122 heads in layers 28–55 of the 64-layer model. The mask-learning procedure and head-set construction are described in Appendix B. We next test the selected heads on full readout continuations.

3.2 GOALSCOPE AND IMPLEMENTATION TRANSFER

arnab: needs a teaser figure.

We use the attention heads identified in 3.1 to build GOALSCOPE, a lightweight probe that verbalizes the sub-goal encoded at a single token position. Building on Patchscope (Ghandeharioun et al., 2024), we cache what the selected heads write to the residual stream at the source pivot and sum these contributions into one vector. A separate run of the same model receives a fixed prompt asking what operation it is performing. We inject the cached vector into this probe's residual stream and generate a natural-language description. The source task reaches the probe through the injected activations; its instructions and code are not supplied as text.

Our main configuration adds the vector at layers 35–45, scaled to 0.1 times the local residual norm. We reuse the same source vector at the probe's last prompt position and every subsequent decoding step, generating the readout greedily. Appendix C gives the exact prompt, normalization, and other design choices to perform the intervention.

Faithfulness. We ask whether the verbalization faithfully describes a nontrivial part of what the LM actually generates as its implementation of the next *meaningful* step. In our controlled setup, we prefill until a target sub-task g_k, giving us a gold target to measure the fidelity of the readouts.

Causality. We inject the same encoding into a code stub in another programming language and ask whether it triggers a nontrivial part of the source operation g_k. For example, an encoding captured from Python is used to steer a C++ continuation. This tests whether the encoding of g_k, captured by the *goal heads*, are abstract from the concrete tokens that implement it and whether this encoding can nudge the LM towards implementing the same sub-task g_k in a new context (C++) with a different sequence of generated tokens.

These two measures expose complementary properties of the encoding: a natural-language description reveals its semantic content, while cross-language transfer shows that it can guide a different realization of the operation. For both measures, we ask an LLM oracle (`claude-sonnet-4-6`) to compare the readout with the extracted LM implementation of the frontier g_k on the source prompt. A positive judgment credits nontrivial agreement, including partial implementations. For *causality* we ensure that the frontier is not aligned with the LM's default behavior to the specific prompt. Exact oracle prompts with additional details are in Appendix D.

3.3 READOUT ACROSS CAPTURE POSITIONS

arnab: doesn't need a different section. illustration of pivot positions + results should be absorbed in 2.

We next examine how compactly the encoding can be read and how the heads contribute to the model's own implementation.

4 GOAL HEADS CARRY AND MEDIATE THE SUB-GOAL

How many heads are needed to read the sub-goal, and how important are these heads for carrying it out? We study both questions: first by varying the number of heads used for readout, then by restricting their access to task information during the model's own code generation.

arnab: Figure 3 and Figure 3 needs to be side-by-side. save space on the main text. maybe Figure 3 should be a bar-chart.

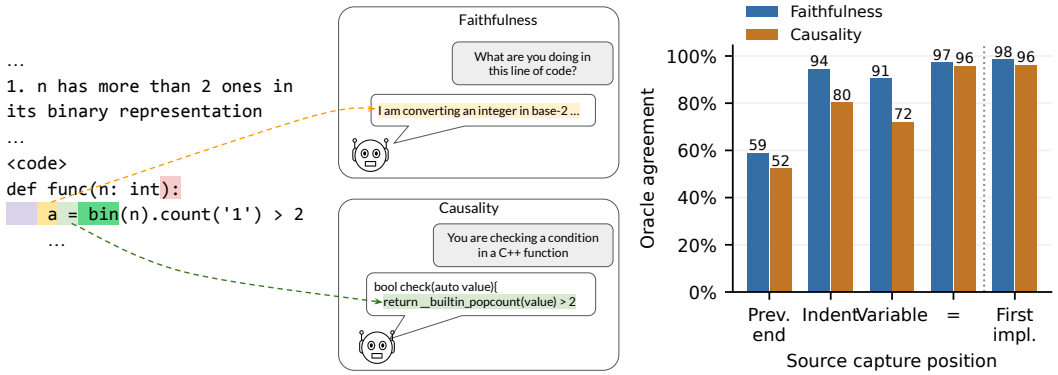


Figure 2: **Readout and implementation transfer across capture positions.** Left: the colored tokens mark the previous-line end, indentation, variable, assignment operator, and first implementation token. A vector captured at one position steers either a brief verbalization (faithfulness) or a C++ continuation (causality). The arrows illustrate two capture sites; both evaluations are run at every position. Right: oracle agreement on the same 253 specifications used in Figure 3, with 122 heads and norm-relative injection at $\alpha = .1$. Only the last position contains an implementation token. The displayed source continuation is context, not input available to earlier pivots.

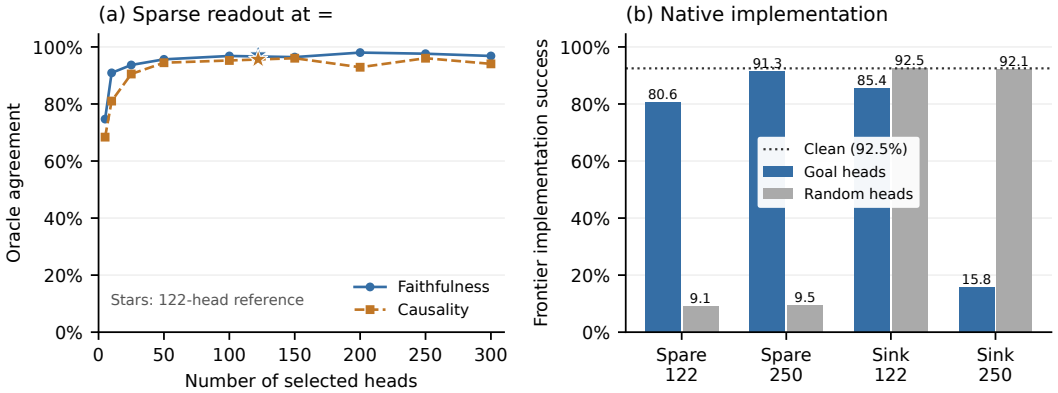


Figure 3: **Compact readout and causal involvement of goal heads.** Left: faithfulness and causality versus head budget at the assignment pivot ($n = 253$, $\alpha = .1$); stars mark the separately selected 122-head reference. Selections are not strictly nested. Right: native frontier implementation success on the same specifications. "Spare" preserves access to the frontier task/code span for the selected heads while blocking other heads in layers 20–63; "sink" restricts the selected heads to key position zero. Random controls receive the corresponding intervention. The dashed line is clean performance (.925). Exact rates and intervention diagrams appear in Appendix F; other capture positions appear in Appendix E.1.

4.1 Small head sets support effective readout

Twenty-five selected heads, approximately 0.98% of the model's 2,560 query heads, achieve 93.7% readout faithfulness and 90.5% implementation transfer at the assignment pivot. The 122-head reference achieves 97.2% and 95.7%, respectively (Figure 3). Much of the sub-goal's semantic content can therefore be recovered from a small fraction of the attention heads.

arnab: recheck the numbers.

Larger selections do not improve performance monotonically. The selections differ in membership and are not strictly nested, so the curves compare available head sets rather than isolate head count alone.

4.2 GOAL HEADS MEDIATE IMPLEMENTATION

We next ask whether the model relies on the *goal heads* to implement the next sub-goal. On the same 253 specifications, we intervene at the assignment pivot and continue the intervention during decoding. We compare the original generation with interventions on the 122-head core, a broader 250-head set, and random-head controls.

Sufficiency of goal-head access. We preserve the selected heads' access to the upcoming task and code span while blocking that access for other heads in layers 20 and above. The blocked span begins just before the frontier description and ends at the final prompt token. Lower layers, MLPs, and attention to positions outside this span operate as usual. This tests whether goal-head access can sustain implementation when other upper-layer attention heads lose access to the same information. The control preserves access for a comparable number of random heads distributed on the upper layers. The exact spans and sampling pools are in Appendix F.

arnab: needs illustration of the ablation setup with a specific prompt, either here or in the appendix

Necessity of goal-head access. In the complementary experiment, we restrict the selected heads to attend only to position zero, the attention sink, while other heads operate normally. The selected heads still contribute the sink value but lose access to the task and code tokens. We compare with applying the same restriction to an equal-size random set, drawn from the depth pool and disjoint from the goal heads.

Figure 3 shows that the LM's competency of implementing the the next sub-goal is largely preserved when only goal heads can access the task information, with most of the heads in deeper layers are blinded. This indicates that the smaller set of 122 goal heads are sufficient in mediating the g_k encoding to later layers for concrete implementation. On the complementary necessity experiment we find that blinding a broader set of 250 goal heads reduces LM competency sharply, confirming that the goal heads are a crucial bottleneck mediating the g_k encoding to the implementation. However, blinding the smaller set of 122 heads keeps the performance largely intact. This means the LM is flexible enough to compensate for the ablation by using other *goal heads* (not in the 122 set) to communicate g_k to later layers.

5 HOW GOAL HEADS ACCESS THE UPCOMING SUB-GOAL

What information do the *goal heads* use to guide the next operation? We examine where the heads attend, how changing their queries or task-span keys and values affects the continuation, and where the sub-goal can be read from the residual stream.

5.1 ATTENTION TO RELEVANT TASK DESCRIPTIONS

The *goal heads* often look back to the task they are about to implement. At a frontier position, their collective value-weighted attention concentrates on tokens in the prompt describing the upcoming predicate. This resembles the *gaze* behavior observed during generation in other settings (Gandikota & Bau, 2026). These qualitative attention patterns suggest a connection between the task descriptions and the goal encoding. We test that connection causally by changing the information available through the heads' keys and values.

5.2 KEY-VALUE TRANSFER OVERWRITES THE FRONTIER

Consider a source prompt with tasks (A, B) and a destination with tasks (A, D), both with A already implemented. The next requested sub-goal is B in the source and C in the destination. We cache the keys and values of the source task tokens and transplant them into the destination's goal-head attention. The destination prompt still requests C; the patched heads now access the source task information B. We ask whether this change makes the model implement B.

arnab: needs concrete illustrative example.

In Figure/Table X, we show 2 scores to evaluate the setup: (i) *causal-shift*, whether the LM implements the source frontier; and (ii) *readout-shift*, whether the GOALSCOPE shifts to the source frontier. This KV-transfer experiment with 250 goal heads show strong generalization across datatypes, even cross-language transfer. The success rate of the KV-transfer in overwriting the destination frontier complements Section 4.2's finding that the goal head set are a crucial bottleneck in the LM's mediation of the sub-goal signal to later layers. The cross-language intervention success also confirm that the

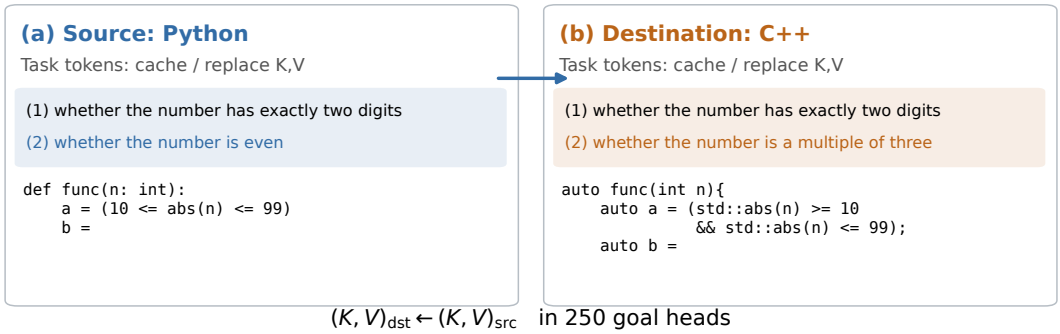


Figure 4: **Task-span KV transfer changes the next operation.** A saved Python-to-C++ example: both prompts have already implemented the shared two-digit check. We replace the keys and values of the destination task span in 250 goal heads with those from the source, whose second task is evenness. The destination still requests divisibility by three, but its patched continuation checks divisibility by two. GoalScope's readout also describes evenness. Chat wrappers and later return statements are omitted; the long C++ prefill line is wrapped for display. The patch is active across prefill and decoding, with no non-goal-head ablation and selected-head OV amplification of 2.

Source \ Destination	Python	C++
Python	99.2 {98.4}	98.4 {93.8}
C++	99.2 {96.1}	100.0 {96.1}

Table 1: **KV transfer across programming languages.** Rows are source languages and columns destination languages. Each cell reports causal success {source readout}, in percent, across four datatypes ($n = 128$ per cell; 512 total). Causal success counts source-only or both-goal verdicts; the readout score counts an exclusively source-directed patched GoalScope description. Both use all examples in the denominator. Neither score certifies full program correctness. Datatype breakdowns and the baseline-valid shift criterion are in Appendix G.

goal heads *read* the task information to encode an abstract representation of the next sub-task g_k, which when patched to a different environment is realized in a different sequence of tokens.

arnab: show the concrete scores, across language confusion table with the format `causality {read-out shift}` here.

5.3 Queries Already Know the Next Sub-Goal

arnab: not yet sure if this should be in the main text

5.4 Reading Intermediate Residual States with OVReader

arnab: no need to show 2 figures on Figure 5. just one figure showing both faithfulness and causality lines together, with standard deviation across the datatypes. maybe pair the figure with the 122 goal head distribution in the model.

At what depth does the upcoming sub-goal become readable? We introduce **OVReader**, which uses the goal heads' value and output projections to read intermediate residual states. Formally, an attention head, its output and value projection read from $OV(h) = W_O W_V \ln(h) = {}^1\!/_{rms(h)} \times W_O W_v(\mu \odot h)$. Qwen-2.5 series of LMs use RMSNorm with μ as the element-wise stretching and ${}^1\!/_{rms(h)}$ is the rescaling to the unit ball. If we ignore the rescaling the OV read becomes, $OV(h) = W_O W_V \text{diag}(\mu)\ h$. $W_O W_V \text{diag}(\mu)$ can be absorbed into a single transformation W_{OV}. For multiple goal heads we can absorb their OV reads into a single transformation $\mathbf{W_{OV}}$, by simply adding each head's OV-transformation matrices.

arnab: add citations to sheridan's work. rename the terms to match exactly. polish the presentation

We apply the aggregated transformation $\mathbf{W_{OV}}$ on a residual state h, and use the output as the steering vector (on Figure X) to get the readout, which we score using the same *faithfulness* and *causality* scores introduced in Section 3.2. Figure 5 shows the readout performance across layers on the " =" token, right before the LM's g_k implementation starts. OVReader readout performance already peaks

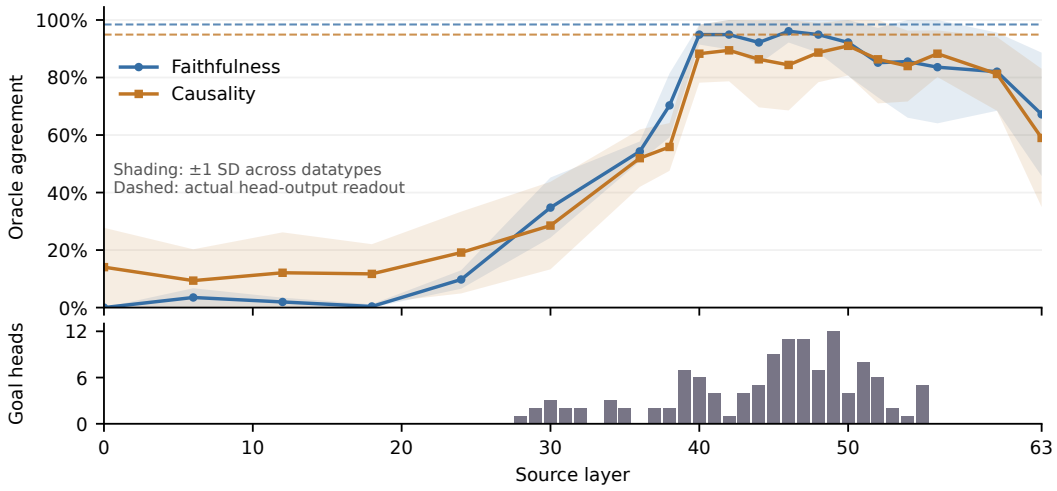


Figure 5: **OVReader at the assignment pivot.** Faithfulness and causality are averaged over four datatypes, with shaded bands showing ± 1 sample standard deviation across datatype rates (64 examples per datatype). Dashed lines show readout from the selected heads' actual outputs on this same cohort. Below: the distribution of the 122 goal heads by layer. These archived 256 examples form a separate cohort from Figure 2; OVReader uses $\alpha = .15$ and the head-output reference uses .1. Causality scores include trivial-readout sanitization. Other capture positions and the evaluated per-head normalization are given in Appendix H.

by layer 40, before the layers where most of the goal heads are concentrated. So, by the time most of the goal heads take action, the next sub-task g_k is already committed and is decodable in the residual latents.

arnab: needs an end.

6 Beyond Explicitly Specified Code Tasks

arnab: didn't check any writing here. will depend on the concrete findings from ongoing experiments.

Do the heads localized on code also carry an operation that the model infers from examples? Our controlled code tasks name each sub-goal in the prompt. We now examine a few-shot setting where the intended relation is conveyed through demonstrations.

Few-shot relation tasks. Following the function-vector setting (Todd et al., 2024), we provide input–output pairs demonstrating a relation, followed by a new input. We retain the 122 heads localized on code and adapt the probe question to ask for the hidden rule in a short phrase. For example, an antonym prompt contains the following pairs:

```
Q: enter  A: exit
Q: feasible  A: infeasible
. . .
Q: mercy  A:
```

The model answers "cruelty" and continues generating question–answer pairs. At the newline following "cruelty," GoalScope reads *"Identifying antonyms."* At the answer-prefix token `A` before a later answer, "slow," it again reads *"Identifying antonyms."* The probe is given neither the name of the relation nor the demonstrations as text. The code-localized head contributions carry enough information to describe the inferred operation.

These are qualitative readouts during the continuing sequence, with the first captured after the initial answer and the second before a later answer. Nearby positions also produce vague or incorrect descriptions. Appendix L gives the full demonstration list, exact probe wording, token indices, and contrasting readouts. The example shows that the code-localized heads can expose an inferred non-code relation; a quantitative evaluation across relations and positions remains to be completed.

Code without enumerated sub-goals. Free-form code poses a related question: can the readout describe a step that the model chooses while solving a broader programming request? Here, evaluation

must identify a meaningful operation in the model's continuation, which may be a local statement or a larger enclosing computation. This extension requires qualitative examples and an explicit choice of sub-goal granularity.

We report replications within other model families and sizes in Appendix J. Further exploratory non-code settings are discussed in Appendix L.

7 Related Work

arnab: didn't check anything here.

Planning and look-ahead in LMs. Prior work shows that autoregressive LMs encode information about future tokens (Pal et al., 2023) and plan toward constraints in later output (Lindsey et al., 2025). We study the upcoming computation at the level of an instrumental sub-goal. Our experiments ask whether its representation can be read before implementation and transferred to guide the same operation in a new context.

claude: Add citations: future-lens / prompt-injection look-ahead, poetry-planning (Anthropic biology), chess/maze planning, emergent world models.

Portable task representations and function abstractions. Function vectors are compact representations of demonstrated input–output tasks, carried by a small set of attention heads and transferable to new contexts (Todd et al., 2024). Concept-induction heads carry word representations that mediate translation across languages (Feucht et al., 2025). Filter heads encode predicates in their query states and reapply them across collections, formats, languages, and tasks (Sen Sharma et al., 2026). Our work examines a related abstraction during multi-step generation: the particular sub-goal the model is about to implement. We localize its encoding in head contributions, verbalize its content, and test its portability across programming languages.

Verbalizing internal representations. Patchscope uses a language model to express information extracted from its hidden states (Ghandeharioun et al., 2024). GoalScope applies this approach to the contributions of causally localized attention heads at a single source position. The cached signal is reused throughout probe generation to obtain a multi-token description or code implementation. OVReader extends this readout to intermediate residual states using the selected heads' value and output projections.

claude: Add: logit lens, tuned lens, SelfIE, function-vector readouts; contrast what GoalScope reads (head-output, multi-token) vs. residual-stream single-token lenses.

High-level interpretability of goal-directed behavior. The high-level interpretability agenda studies model behavior through abstractions that can be investigated without a complete account of every underlying computation (Greenblatt, 2024). We use an instrumental sub-goal as such an abstraction. Cross-language transfer tests whether its encoding can guide different implementations, while restrictions on goal-head attention test its role in the model's own generation.

claude: Cite causal abstraction / DAS (Geiger et al.), high-level causal mediation; position our cross-language transfer as the abstraction test.

8 Discussion

arnab: didn't check anything. will depend on the concrete framing and findings.

We find that code LLMs carry a compact representation of their upcoming sub-goal before writing its implementation. The representation can be extracted from attention-head contributions, verbalized in natural language, and transplanted to trigger the same operation in another programming language. The heads also mediate the model's own implementation: preserving their access to task information largely preserves performance, and restricting a broader set sharply reduces it. These findings connect an interpretable description of what the model is about to do with an internal representation that can influence its behavior.

An abstraction of the upcoming operation. The portability of the encoding suggests a separation between the identity of a sub-goal and its realization in code. An operation extracted from Python can guide a C++ implementation, even though the two require different tokens. Natural-language verbalization provides a complementary way to inspect this semantic content. Our evidence concerns the upcoming operation near the start of its implementation; it leaves open how much of a longer plan is represented and how its details are chosen. Verbalizability also does not establish that the representation's native format is natural language.

From goal representation to goal selection. The intervention results identify several parts of goal-directed computation. Queries carry information that can favor a particular sub-goal; task-span keys and values supply information to which the model can adapt; and the heads' projections expose the upcoming operation in intermediate residual states. The ablations further distinguish a small set

sufficient for readout from a broader set important for implementation. These observations constrain the mechanism, while leaving open how the model combines completed work with the specification to select its next step. Independently varying progress, task order, and target identity offers a way to investigate that computation. The current interventions establish causal involvement under specific access restrictions, rather than a unique bottleneck or an isolated goal-selection circuit.

Scope and limitations. Our principal quantitative setting uses explicit task specifications and a known frontier. The oracle measures nontrivial semantic agreement and can credit partial implementations, so success rates should be interpreted separately from executable program correctness. Capture positions are repeated observations of the same specifications, and head-budget comparisons use selections that are not strictly nested. Replications use each model's own heads and activations; differences between models combine checkpoint, head-selection, and generation effects. The few-shot example is qualitative, and evaluating free-form code requires further examples and decisions about sub-goal granularity. These settings are promising tests of how far the code-localized representation generalizes.

GOALSCOPE provides a concrete tool for examining what a model is about to do as generation proceeds. Further work can use these readouts to study how sub-goals change, when an intended operation fails to be implemented, and how the model decides to move on to the next step. Our exploratory boundary-signal interventions (Appendix M) provide a complementary starting point for the last question.

ETHICS

REPRODUCIBILITY

ACKNOWLEDGEMENTS

REFERENCES

Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, et al. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 1(1):12, 2021.

Sheridan Feucht, Eric Todd, Byron C Wallace, and David Bau. The dual-route model of induction. In *Conference on Language Models (COLM)*, 2025. URL https://openreview.net/forum?id=bNTrKqqnG9.

Rohit Gandikota and David Bau. Gaze heads: How vlms look at what they describe. *arXiv preprint arXiv:2606.14703*, 2026.

Asma Ghandeharioun, Avi Caciularu, Adam Pearce, Lucas Dixon, and Mor Geva. Patchscopes: A unifying framework for inspecting hidden representations of language models. In *International Conference on Machine Learning (ICML)*, 2024.

Ryan Greenblatt. High-level interpretability: how LLMs make decisions in pursuit of goals, 2024. Interview remarks; high-level, intervention-style interpretability of goal-directed behavior.

Jack Lindsey, Wes Gurnee, Emmanuel Ameisen, Brian Chen, Adam Pearce, Nicholas L. Turner, Craig Citro, Joshua Batson, et al. On the biology of a large language model. *Transformer Circuits Thread*, 2025. URL https://transformer-circuits.pub/2025/attribution-graphs/biology.html.

Aaron Mueller, Jannik Brinkmann, Millicent Li, Samuel Marks, Koyena Pal, Nikhil Prakash, Can Rager, Aruna Sankaranarayanan, Arnab Sen Sharma, Jiuding Sun, et al. The quest for the right mediator: Surveying mechanistic interpretability for nlp through the lens of causal mediation analysis. *Computational Linguistics*, 52(1):331–378, 2026.

Koyena Pal, Jiuding Sun, Andrew Yuan, Byron C. Wallace, and David Bau. Future lens: Anticipating subsequent tokens from a single hidden state. In *Conference on Computational Natural Language Learning (CoNLL)*, 2023.

Arnab Sen Sharma, Giordano Rogers, Natalie Shapira, and David Bau. LLMs process lists with general filter heads. In *International Conference on Learning Representations (ICLR)*, 2026. URL https://filter.baulab.info/. arXiv:2510.26784.

Eric Todd, Millicent L. Li, Arnab Sen Sharma, Aaron Mueller, Byron C. Wallace, and David Bau. Function vectors in large language models. In *International Conference on Learning Representations (ICLR)*, 2024.

Jesse Vig, Sebastian Gehrmann, Yonatan Belinkov, Sharon Qian, Daniel Nevo, Yaron Singer, and Stuart Shieber. Investigating gender bias in language models using causal mediation analysis. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (eds.), *Advances in Neural Information Processing Systems*, volume 33, pp. 12388–12401. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/file/92650b2e92217715fe312e6fa7b90d82-Paper.pdf.

A Evaluation Corpus and Exact Prompt

A.1 Locked prefill-to-frontier corpus

The main GoalScope evaluation uses `data/goalscope_eval.json`, generated with seed 123 from the QTASKS predicate bank. It contains 253 Python specifications. Each specifies two, three, or four predicates and prefills one or more completed predicates before the frontier assignment. The function and intermediate-variable naming conventions are fixed; the exact example below uses `func` and successive letter variables. Sampling balances the frontier predicate within each datatype, rather than making the four datatype counts identical. The number of completed predicates is between one and three and always less than the total number of predicates.

Datatype	Specs	Frontiers	Repeats	$T = 2$	$T = 3$	$T = 4$	Records/mode
int	65	13	5	26	15	24	325
list[int]	60	12	5	23	25	12	300
str	64	16	4	26	21	17	320
list[str]	64	16	4	17	21	26	320
Total	253	57	—	92	82	79	1265

Table 2: Distribution of the locked evaluation corpus. T is the number of predicates in the specification; repeats count distinct prefill contexts per frontier. Frontier identities are counted within datatype. Each specification contributes five capture positions per evaluation mode.

Datatype	1 completed	2 completed	3 completed
int	38	18	9
list[int]	36	19	5
str	43	16	5
list[str]	34	20	10
Total	151	73	29

Table 3: Number of completed predicates in the prefilled context. These counts are specification-level, not token-position-level.

The frontier bank excludes the integer predicates `is_negative`, `is_positive`, and `sign`, and the integer-list predicates `all_positive`, `contains_zero`, `count_negative`, and `count_positive`. This exclusion applies to measured frontier targets; it does not forbid such predicates in completed context. The following lists give every retained frontier ID; each occurs the number of times specified in Table 2.

int (5 contexts per frontier). `absolute_value`, `count_set_bits`, `digit_count`, `digit_sum`, `is_even`, `is_multiple_of_3`, `is_odd`, `is_palindrome`, `is_perfect_square`, `is_power_of_two`, `is_prime`, `is_two_digit`, `last_digit`.

list[int] (5 contexts per frontier). `count_even`, `count_odd`, `distinct_count`, `has_duplicate`, `has_even`, `is_sorted`, `max_val`, `mean_int`, `min_val`, `product_all`, `total_sum`, `value_range`.

str (4 contexts per frontier). `all_lowercase`, `count_digits`, `count_letters`, `count_lowercase`, `count_spaces`, `count_uppercase`, `first_is_letter`, `has_digit`, `has_space`, `is_alnum`, `is_palindrome`, `last_char`, `length`, `starts_upper`, `vowel_count`, `word_count`.

list[str] (4 contexts per frontier). `all_unique_lengths`, `any_empty`, `count_all_lowercase`, `count_capitalized`, `count_ends_consonant`, `count_long`, `count_short`, `count_starts_vowel`, `count_with_digit`, `distinct_count`, `has_palindrome`, `longest_word_length`, `shortest_word_length`, `total_length`, `total_uppercase_letters`, `total_vowels`.

```
<system>
You are a helpful assistant proficient in programming. Always wrap your code in
<code></code> tags. Just the code, no need to comment or explain.
</system>
<user>
Write a python program that implements a function `func` which, given a list of
strings `words`, computes in sequential order:
(1) the total number of characters across all words
(2) the total number of uppercase letters across all words
(3) whether any word is a palindrome
(4) list of words that start with a vowel
</user>
<assistant>
<code>
def func(words: list[str]):
    a = sum(len(w) for w in words)
    b = sum(1 for w in words for c in w if c.isupper())
    c =
```

Figure 6: **Illustrative task prompt.** The assistant response is prefilled up to the target frontier (blue), here a palindrome check after character and uppercase counts. Generic `<system>`, `<user>`, and `<assistant>` tags stand in for the model family's chat tokens. The input ends at the final assignment operator. This illustrative prompt is distinct from the exact saved evaluation example below.

A.2 Exact source prompt and capture positions

The following is the complete saved prompt for `int_python_000` in the Qwen2.5-Coder-32B evaluation, including chat delimiters and the assistant prefill. The input ends immediately after the final = without an added newline. In the prompt illustrations, \n marks an actual newline; unmarked wrapping only fits the text to the box. The accompanying text files preserve the exact bytes.

```
<|im_start|>system\n
You are a helpful assistant proficient in programming. Always wrap your code inside
<code></code> blocks. Just the code, no need to comment or explain.<|im_end|>\n
<|im_start|>user\n
Write a Python program that implements a function `func` which, given an integer `n`,
computes in sequential order:\n
(1) the number of digits\n
(2) whether the number is even\n
<|im_end|>\n
<|im_start|>assistant\n
<code>\n
def func(n: int):\n
    a = len(str(abs(n)))\n
    b =
```

The first predicate, digit count, is already implemented; the frontier is evenness. The saved capture indices are 102 (previous-line end), 103 (indentation), 104 (variable), 105 (assignment operator), and 106 (first implementation token). They are indices in this model's tokenization, not universal character offsets. The saved clean continuation is:

```
 (n % 2 == 0)\n
    return a, b\n
</code>
```

The gold implementation span used by the judge is `b = (n % 2 == 0)`. More generally, the runner extracts the actual generated frontier assignment as `impl_span`; it does not use the natural-language predicate description as the oracle's CODE field. This tests agreement with the generated operation

and must be distinguished from clean correctness against the requested predicate. The controlled frontier identifies the assignment directly, without an oracle segmentation call.

Five capture positions yield 1,265 records per mode and model, with repeated observations of the same 253 specifications. Rates pool records unless a position or datatype is explicitly selected. This corpus is separate from the 64-per-datatype OVReader sampling and the source/destination-pair corpora used for Q/KV transfer; their counts should not be combined.

B LOCALIZING ATTENTION HEADS

Localization uses contrastive single-predicate prompts and learns a mask jointly over attention heads. The target is a token appropriate for the source predicate in the destination language, chosen to differ from the source continuation token. Applying the learned mask selects head activations for replacement. Subsequent readout uses the selected heads' projected contributions; the localization next-token objective and the multi-token evaluation are distinct.

The core Qwen2.5-Coder-32B selection has 122 heads in layers 28–55, from a thresholded mask within the candidate layer band. The broader set used in ablations contains 250 heads and includes earlier and later layers. The small-set readout sweep uses available selections from training snapshots and mask sweeps; these are not strictly nested prefixes of one ranking. Counts alone therefore do not determine head membership or the intervention's depth distribution.

Draft completion item: consolidate localization training splits, mask optimization settings, selection-file hashes, and per-model selection provenance into a final configuration table.

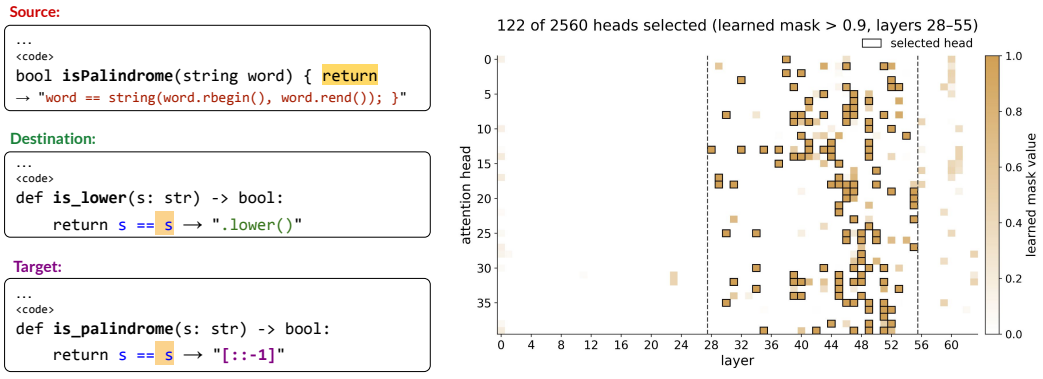


Figure 7: **Locating the goal heads.** Left: source and destination prompts request different predicates in different programming languages. A separate target prompt requests the source predicate in the destination language. The two destination-language implementations share a prefix and differ in the next token. We patch source head contributions into the destination using the mask-weighted intervention in Equation (2), and jointly train the masks to promote the target continuation. Here, Python's target token `[` differs from the source continuation `word`, discouraging direct token copying. Right: the learned mask and the selected 122 heads in Qwen2.5-Coder-32B. The target prompt determines the optimization target; it is not supplied to the patched destination.

C GOALSCOPE

C.1 CAPTURE POSITIONS

The evaluation indexes the final four prompt tokens around a prefilled frontier assignment and the first generated implementation token: previous-line end, indentation, variable, assignment operator, and first implementation. Position names refer to the actual tokenized boundary; they are not interchangeable with an entire source line. The first implementation token is not a pre-implementation pivot. A cached state at a given position has no causal access to subsequent tokens, even when those tokens are displayed for context.

C.2 AGGREGATION AND PROBE GENERATION

At source position t, the main readout aggregates projected head contributions $g_t = \sum_{(\ell,j)\in H} a_t^{\ell j}$. At probe layer ℓ and current position s, norm-relative additive injection uses

$$r_{\ell,s} \leftarrow r_{\ell,s} + \alpha \|r_{\ell,s}\| \frac{g_t}{\|g_t\|}. \tag{3}$$

The main configuration uses $\alpha = .1$, layers 35–45 inclusive, and repeated injection during greedy probe generation. This coefficient is a fraction of the probe's local residual norm and is not in-

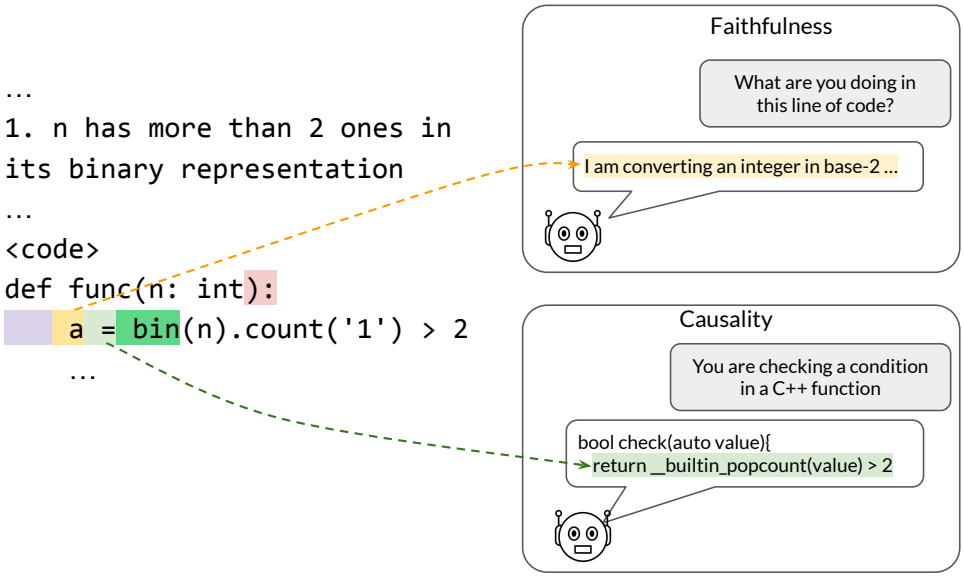


Figure 8: **Capture positions and the two probes, enlarged.** Pink marks the previous-line end, purple indentation, yellow the variable, light green the assignment operator, and dark green the first implementation token. The orange and green arrows illustrate faithfulness and causality readouts from the variable and assignment positions, respectively. In the evaluation, each probe is run separately at all five positions. Only source activations up to the selected pivot enter its cached vector.

terchangeable with an unnormalized additive coefficient. The source vector is fixed for the whole readout.

The default verbalization prompt is:

> System: You are a helpful assistant proficient in programming.
> User: What are you doing at this line of the code. Keep your answer as brief as possible.
> Assistant:

The probe receives no source code as text. Implementation transfer uses a code-stub probe instead and measures nontrivial agreement with the source subgoal. A distinct replacement variant substitutes selected heads' pre-output-projection activations in their corresponding layers, rather than aggregating them into a single direction; this is the variant used in the cross-model table. Its layer sites come from the head set, not the additive injection band.

C.3 Evaluation

An oracle judges whether the generated description or implementation matches a nontrivial part of the extracted frontier code. The full system rubrics, user-message templates, and saved examples are in Appendix D. Reported implementation-transfer results apply trivial-readout sanitization to avoid crediting a probe's default sign/zero comparisons. These judgments do not certify execution correctness. Clean continuation agreement, probe readout agreement, and intervention-induced changes are separate measurements. Corpus size and capture-position pooling are stated with each result.

D Exact Oracle Prompts and Scoring

The evaluation calls `claude-sonnet-4-6` with a system rubric and a user message, a maximum response budget of 512 tokens, and caching enabled. The complete system prompts below are extracted verbatim from the runner. Their MD5-prefix rubric identifiers, `78bfd5f4` (faithfulness) and `9c106246` (causality), match the saved configurations used here. The code's mode name *causality*

denotes the implementation-transfer metric in the main text. Spelling, punctuation, and lenient scoring language are preserved as used, rather than edited for exposition.

As in Appendix A, \n marks actual newlines in the illustrated prompts; soft wrapping is unmarked.

D.1 FAITHFULNESS: SYSTEM MESSAGE

```
You are grading a lightweight interpretability probe (a patchscope). The DESCRIPTION
below was read out of a model's hidden state at a SINGLE token while the model was
writing one line of Python CODE that checks some property of its input. Because it is
squeezed out of one token under a steering/patching setup, the description is usually
NOISY: it may be wrapped in chat scaffolding (like 'Sure, but first let us ...'),
contain code fences or half-written code, be cut off mid-sentence, or repeat and loop.
Look PAST all of that surface noise and grade only the SUBSTANCE.\n
\n
The question: does the description surface ANYTHING genuinely relevant to what the CODE
is doing — any relevant operation, quantity, sub-step, or property of the input? Pulling
even a small correct fragment out of a single token is a SUCCESS, so grade as leniently
as possible — EXCEPT that a read-out which has collapsed into a degenerate repeating
loop is NOT a success, no matter what tokens the loop contains (see HEAVY_LOOP).\n
\n
Decide in this order — check MALFORMED and HEAVY_LOOP FIRST, and only then YES / NO /
NO_INFO:\n
\n
  MALFORMED  - the description is contentless: empty, pure punctuation or garbage, or
  repetition that carries NO relevant meaning.\n
  HEAVY_LOOP - the description is DOMINATED by a short repeating cycle: a unit of
  roughly five words/tokens or fewer that repeats over and over (e.g. 'value >= value
  >= value >= ...', 'len len len len', 'sum sum ( sum % sum == 0' on repeat, 'x % x == x
  % 0' on repeat). Once you collapse the repetition, little or no coherent description
  is left. Assign HEAVY_LOOP EVEN IF the looped fragment contains a relevant operator,
  name, or token — a stuck loop is not a faithful read-out. This takes PRECEDENCE over
  YES.\n
  YES        - the description contains SOME intelligible content genuinely relevant to
  what the code does, however partial, imprecise, or garbled — and it is NOT merely a
  short repeating loop.\n
  NO         - the description is coherent and about a specific thing, but that thing is
  COMPLETELY UNRELATED to what the code does. No conceptual sub-parts have been
  identified correctly.\n
  NO_INFO    - the description is coherent but too generic to indicate any specific
  aspect of the code (e.g. just 'doing a computation on the number').\n
\n
The following do NOT count against the description (still YES, provided it is not a
HEAVY_LOOP):\n
  - Wrong specifics: a wrong constant, threshold, bound, modulus, operator, or
  comparison. If the code computes some quantity and compares it to a value, naming the
  quantity alone is enough and the exact value can be wrong or missing. Example: the
  code implements `average temperature > 30`; a read-out about `computing the average
  temperature` is YES even though the `> 30` part is wrong or absent.\n
  - Wrong implementation nuance: naming a different mechanism for the same idea.
  Example: the read-out says `sorting with bubble sort` while the code calls .sort() or
  uses quicksort -> still YES.\n
  - Partial coverage: naming just one relevant ingredient of a multi-part check
  counts.\n
\n
Grade on meaning, not on shared keywords. Output only YES, NO, NO_INFO, MALFORMED, or
HEAVY_LOOP.\n
Put your final answer inside <verdict>{ans}</verdict> tags, with no other text or
explanation, to facilitate automated parsing.
```

User-message template. Braced fields below denote runtime substitutions, not literal strings sent to the oracle. `impl_span` is the extracted Python frontier assignment; `readout.strip()` removes leading and trailing whitespace from the generated probe readout.

```
Code: {impl_span}\n
Description: {readout.strip()}\n
Answer:
```

Exact instantiated example. This user message is copied from the saved assignment-pivot record for `int_python_000` in the 122-head norm-relative additive run.

```
Code: b = (n % 2 == 0)\n
Description: I'm checking if the number is even by using the modulus operator (`%`). If
`number % 2 == 0`, it's even.\n
Answer:
```

D.2 CAUSALITY / IMPLEMENTATION TRANSFER: SYSTEM MESSAGE

```
You are grading a lightweight interpretability probe (a patchscope). The DESCRIPTION
below was read out of a model's hidden state at a SINGLE token while the model was
writing one line of Python CODE that checks some property of its input. The read-out
method is a steering setup on a informationless C++ function stub. Because it is
squeezed out of one token, the description is usually NOISY: it may be half-written
code, be cut off mid-generation, be syntactically incorrect, or repeat and loop. Look
PAST all of that surface noise and grade only the SUBSTANCE.\n
\n
The question: does the C++ completion surface **ANYTHING GENUINELY RELEVANT** to what
the CODE is doing — any relevant operation, quantity, sub-step, or property of the
input? Pulling even a small correct fragment should be considered a SUCCESS, so grade
leniently — EXCEPT that a completion which has collapsed into a degenerate repeating
loop is NOT a success, no matter what tokens the loop contains (see HEAVY_LOOP).\n
\n
Decide in this order — check MALFORMED and HEAVY_LOOP FIRST, and only then YES / NO /
NO_INFO:\n
\n
  MALFORMED  - the completion is contentless: empty, pure punctuation or garbage, or
  repetition that carries NO relevant meaning.\n
  HEAVY_LOOP - the completion is DOMINATED by a short repeating cycle: a unit of
  roughly five words/tokens or fewer that repeats over and over (e.g. 'value >= value
  >= value >= ...', '(value(value(value...', 'sum sum ( sum % sum == 0' on repeat,
  'value == sum sum sum value ==' on repeat). Once you collapse the repetition, little
  or no coherent content is left. Assign HEAVY_LOOP EVEN IF the looped fragment
  contains a relevant operator, name, or token — a stuck loop is not a successful
  transfer. This takes PRECEDENCE over YES.\n
  YES        - the completion contains SOME intelligible content genuinely relevant to
  what the code does, however partial, imprecise, or garbled — and it is NOT merely a
  short repeating loop.\n
  NO         - the completion is coherent and about a specific thing, but that thing is
  COMPLETELY UNRELATED to what the code does. No conceptual sub-parts have been
  identified correctly.\n
  NO_INFO    - the completion is coherent but too generic to indicate any specific
  aspect of the code (e.g. just 'doing a computation on the number').\n
\n
The following do NOT count against the completion (still YES, provided it is not a
HEAVY_LOOP):\n
  - Wrong specifics: a wrong constant, threshold, bound, modulus, operator, or
  comparison. If the code computes some quantity and compares it to a value, naming the
  quantity alone is enough and the exact value can be wrong or missing. Example: the
  code implements `count(word, "b") > 5`; a c++ completion that counts the number of a
  `substr` in the `word` should be judged YES, even if the substr != b and/or the
  threshold is wrong.\n
  - Partial coverage: naming just one relevant ingredient of a multi-part check counts.
  For `count(word, "b") > 5`, a c++ completion that only counts the number of `substr`
  in the `word` is still YES, even if it doesn't check the threshold.\n
```

```
  - Different implementation nuance: different implementation strategies for the same
  idea. Example: the python code implements nth fibonacci recursively; a c++ completion
  implements it iteratively. Still YES, because the underlying property is the same.\n
\n
Finally, grade on meaning, not on shared keywords. Output only YES, NO, NO_INFO,
MALFORMED, or HEAVY_LOOP.\n
Put your final answer inside <verdict>{ans}</verdict> tags, with no other text or
explanation, to facilitate automated parsing.
```

User-message template. The C++ `readout` includes the fixed assistant prefill `bool check(auto value){` concatenated with the generated continuation before grading.

```
Python (gold): {impl_span}\n
C++ (read-out): {readout.strip()}\n
Answer:
```

Exact instantiated example. The following is the saved user message for the same specification and assignment pivot in the implementation-transfer run.

```
Python (gold): b = (n % 2 == 0)\n
C++ (read-out): bool check(auto value){ return (value % 2 == 0); }\n
Answer:
```

D.3 Generation prompt for the implementation probe

This is the role-separated probe instruction and assistant prefill, before the model-specific chat template is applied. It is distinct from the judging prompts above.

```
System:\n
You are a helpful assistant proficient in programming.\n
\n
User:\n
You are implementing a simple C++ function that checks for a condition. Just write the
function body, without any comments or explanations. Assume all the necessary headers
are already included.\n
\n
Assistant (prefill):\n
bool check(auto value){
```

D.4 Verdict parsing and deterministic exclusions

The parser first looks for a verdict inside `<verdict>` tags and otherwise uses the last recognized bare verdict. Unrecognized responses become `UNPARSED`. The reported hard-YES rate counts only `YES` in the numerator and all records in the denominator; `NO`, `NO_INFO`, `MALFORMED`, `HEAVY_LOOP`, and `UNPARSED` are not successes. These rubrics explicitly permit partial content and wrong constants or thresholds. Consequently, their rates are not exact predicate accuracy or executable-program accuracy.

Before calling the oracle, empty readouts become `MALFORMED`. In the causality mode, a deterministic sanitizer assigns `NO` to continuations containing only the generic function scaffolding and sign/zero comparisons. The implementation removes the recognized scaffolding words, logical/comparison operators, standalone zero literals, and punctuation; a readout is trivial if nothing remains. Genuine arithmetic or bitwise operations and nonzero constants survive this test. Such short-circuited records have no oracle user message. This readout exclusion is distinct from the frontier exclusions in Appendix A.

Injected source	Faithfulness	Implementation transfer
122-head aggregate	.879	.792
Attention output, L40–50	.343	.334
Residual, L25–56	.031	.020
Residual, L40–50	.009	.006

Table 4: Source comparison under norm-relative injection at $\alpha = .1$ on the locked corpus, pooled over five positions. Poor raw-residual injection is a result about this readout configuration, not proof that the residual lacks subgoal information.

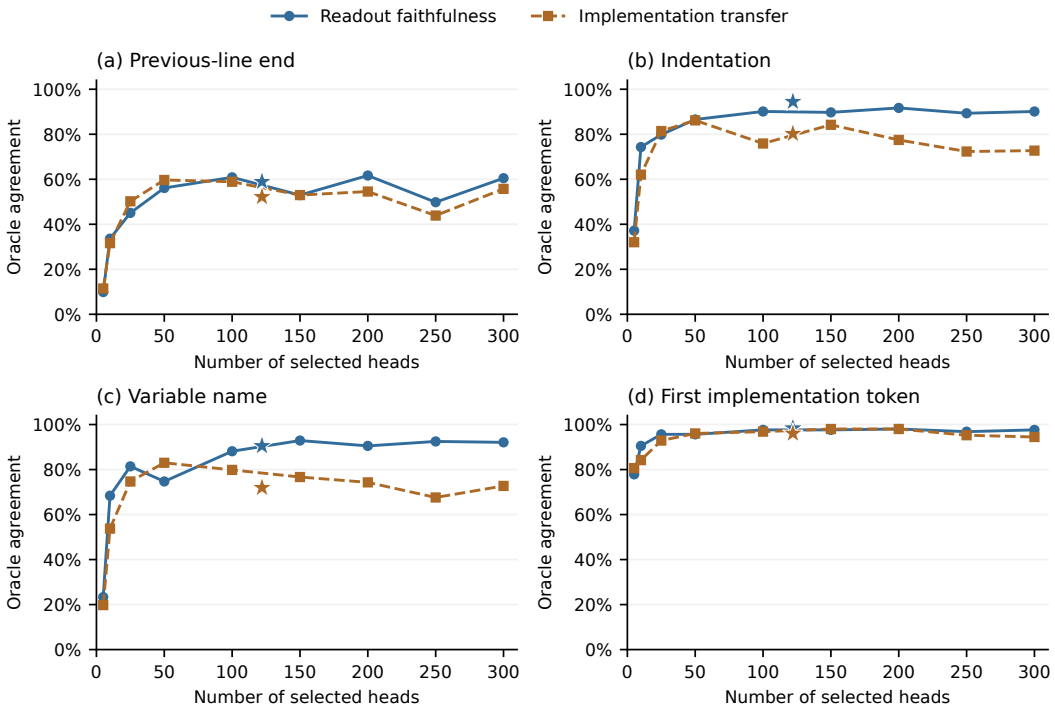


Figure 9: Head-budget curves for the remaining four capture positions, using the same 253 specifications and configurations as Figure 3. Stars indicate the 122-head reference; blue solid lines show readout faithfulness and orange dashed lines implementation transfer. The first implementation token is a comparison position, not evidence of encoding before implementation. The common vertical scale makes differences across positions visible.

E Readout Sources and Head Budgets

Selected contributions provide a substantially more effective injection direction than the tested full-attention and raw-residual aggregates. Equal relative norm controls overall perturbation magnitude, but does not ensure that a shared injection strength or aggregation rule is optimal for every source. OVReader provides a separate test of information extracted from the residual.

E.1 Head-budget results at other capture positions

We recompute each rate as the fraction of saved oracle verdicts equal to `YES`, including all 253 records per position in its denominator. The same specification identifiers occur at every head budget in both evaluation modes. Each position is a repeated measurement of those specifications. All runs use norm-relative additive injection at $\alpha = .1$, layers 35–45, and steering throughout probe decoding. The head selections differ in construction and are not strictly nested; the 122-head reference is displayed separately from the connected sweep.

F Details of Causal and Context Interventions

The native ablation evaluation uses 253 frontier assignments. Both intervention types begin at the final prompt token and continue at subsequent query positions during decoding; earlier query rows remain unmodified. For span restriction, the blocked key interval begins one token before the frontier description and extends through the prompt's final assignment token. Heads in layers 20–63 outside the spared set are blocked from that interval. For sink restriction, selected heads attend only to key position zero. The latter produces the sink value contribution and is not mathematically identical to zeroing a head.

The 122-head core and 250-head set differ in both membership and depth coverage. Random controls sample from the corresponding depth pool, not a matched count at each individual layer. For a set containing heads below layer 20, those heads are outside the span-restriction band. Thus total selected count and number of spared heads inside that band must be distinguished. The experiments test these specified interventions, not arbitrary removal of the entire complement or isolation of the selected heads from the rest of the network.

Condition	122 heads	250 heads
Clean baseline	.925	.925
Spare goal-head access	.806	.913
Spare random-head access	.091	.095
Restrict goal heads to sink	.854	.158
Restrict random heads to sink	.925	.921

Table 5: Frontier implementation success under span restriction and sink-attention interventions ($n = 253$). "Spare" preserves selected heads' access to the blocked task/code span in layers 20 and above. "Restrict" limits the selected heads to attention at position zero.

For the 122-head sufficiency run, both goal and random conditions block 1,638 heads in layers 20–63, leaving 122 with access in that band. For the broader run, the saved configurations block 1,524 heads in the goal condition and 1,590 in the random condition, leaving 236 and 170, respectively. Thus the broader sufficiency control does not exactly match the number of spared heads inside the intervention band. The sink comparisons use the stated selected-set sizes.

	122 heads		250 heads	
Blocked keys in selected heads	Goal	Random	Goal	Random
Task descriptions	.881	.925	.806	.925
Task descriptions and progress	.372	.925	.249	.925
All except sink	.854	.925	.158	.921

Table 6: Additional necessity interventions from the saved 253-specification evaluation. The clean baseline is .925 throughout. Restricting task descriptions alone has a smaller effect than restricting task and progress tokens together. These are access interventions at the current and subsequent query positions.

(a) Sufficiency: spare goal-head access

```
<system>
You are a helpful assistant proficient in programming. Always wrap
your code in <code></code> tags. Just the code, no need to comment
or explain.
</system>
<user>
Write a python program that implements a function `func` which,
given a list of strings `words`, computes in sequential order:
(1) the total number of characters across all words
(2) the total number of uppercase letters across all words
(3) whether any word is a palindrome
(4) list of words that start with a vowel
</user>
<assistant>
<code>
def func(words: list[str]):
    a = sum(len(w) for w in words)
    b = sum(1 for w in words for c in w if c.isupper())
    c =
```

(b) Necessity: restrict goal heads to position zero

```
<bos><system>
You are a helpful assistant proficient in programming. Always wrap
your code in <code></code> tags. Just the code, no need to comment
or explain.
</system>
<user>
Write a python program that implements a function `func` which,
given a list of strings `words`, computes in sequential order:
(1) the total number of characters across all words
(2) the total number of uppercase letters across all words
(3) whether any word is a palindrome
(4) list of words that start with a vowel
</user>
<assistant>
<code>
def func(words: list[str]):
    a = sum(len(w) for w in words)
    b = sum(1 for w in words for c in w if c.isupper())
    c =
```

Figure 10: **Ablation setups.** (a) Only spared heads can attend to the green span; other heads in layers 20–63 are blocked from it. (b) Selected heads cannot attend to the orange span and instead attend to position zero, denoted by generic <bos> (<|im_start|> in Qwen). Purple boxes mark the final `=` query. The restriction continues at subsequent generation positions; earlier query rows are unchanged. Generic role tags stand in for model-specific delimiters.

Condition	Continuation after `b =`	Oracle verdict
Clean	`(n % 2 == 0)`	YES
Spare goal-head access	`(n % 2 == 0)`	YES
Spare random-head access	`(n * (n + 1)) // 2`	NO
Restrict goal heads to sink	`nn``	MALFORMED
Restrict random heads to sink	`(n % 2 == 0)`	YES

Table 7: **An observed effect of ablation.** Exact saved continuations for `int_python_000`, whose prompt appears in Appendix A. The frontier is evenness after digit count. Intervention rows use the broader 250-head configuration. This selected example illustrates the aggregate results.

G TASK AND QUERY TRANSFER

Task-span KV transfer uses per-query-head output routing to avoid transferring to every query head sharing a KV head. Task spans are padded to equal length and source keys are re-rotated to destination positions; equal lengths do not imply identical absolute starts. The KV patch is active across prefill positions and decoding. Query transfer patches the specified pivot tokens and reuses the source pivot query during the steering window (32 forward passes in the evaluation CLI, including prefill). The reported transfer configurations also amplify selected OV contributions with coefficient 2; this differs from the GoalScope probe's coefficient 0.1.

The principal multi-predicate 250-head runs report approximately 94% source-directed generation for query transfer ($n = 2048$) and 99% for KV transfer at the assignment pivot ($n = 512$), spanning four datatypes and four language directions. The latter uses no non-goal-head ablation. These rates count source-or-both oracle labels and do not imply perfect program correctness. GoalScope's patched source readout is a separate outcome. Q-filter examples exhibit readout/behavior disagreement; aggregate readout success should not be treated as a substitute for implementation success.

The qualitative task-order example in Figure 12 uses unequal source/destination progress. It should not be described as an identical-progress experiment. Broader conclusions about query content require a systematic crossed design. We make no claim that these interventions demonstrate a particular set-subtraction operation.

(a) Source run, $M(p_{\text{src}})$

```
...
returns true if the shortest word starts with a vowel
...
<code>
auto func(std::vector<std::string> words){

    std::vector<std::string> sorted_words = words;
    std::stable_sort(sorted_words.begin(),
sorted_words.end(), [](const std::string& a, const
std::string& b){ return a.size() < b.size(); });
    char first_char = std::tolower(sorted_words[0][0]);
    return first_char == 'a' || first_char == 'e' ||
first_char == 'i' || first_char == 'o' || first_char == 'u';
}
```

↳ **readout** ' ' => *I'm checking if the string starts with a vowel.*

(b) Destination run, $M(p_{\text{dst}})$

```
...
returns true if the longest word is a palindrome
...
<code>
def func(words: list[str]):
    res = sorted(words, key=len)
    return res[-1] == res[-1][::-1]
</code>
```

↳ **readout @** ' return' => *I'm checking if the string is equal to its reverse.*

(c) Patched run, $M(p_{\text{dst}} | \leftarrow \{K, V\}_{\text{src}})$

```
...
    return res[0][0].lower() in 'aeiou'
</code>
```

↳ **readout @** ' return' => *I'm checking if the first character of the string is in 'aeiouAEIOU'.*

Figure 11: **KV transfer after a shared initial sub-task.** The source and destination both sort the words by length (blue), then pursue different tasks: a vowel test on the shortest word versus a palindrome test on the longest word. Replacing the destination task-span keys and values available to the goal heads with those from the source changes the continuation to a vowel test on the shortest word. The readout shifts accordingly. Source, native destination, and patched continuations are red, green, and purple; yellow marks their pivots. This is an illustrative slide example, separate from the scored multi-predicate cohort. In the evaluated implementation, the fixed source span is routed to selected query heads at every forward pass; other heads retain destination KV access.

G.1 KV TRANSFER BY DATATYPE

The braced score counts patched descriptions judged exclusively source-directed, with all examples in the denominator. The stored `valid` flag asks whether the original source and destination readouts match their respective frontiers, allowing a both-goal verdict; its rates are 99.2%, 96.9%, 96.1%, and 94.5% for Python–Python, Python–C++, C++–Python, and C++–C++, respectively. The stored `shifted` flag counts patched source-only or both-goal readouts, without requiring either clean readout to be correct. Its unconditional rates happen to match the braced scores here because none of these patched readouts is labeled both. The evaluation script also reports shift conditional on `valid`; that denominator differs from the unconditional scores shown here.

int.

Source \ Destination	Python	C++
Python	100.0 {96.9}	93.8 {96.9}
C++	100.0 {100.0}	100.0 {96.9}

list[int].

Source \ Destination	Python	C++
Python	96.9 {100.0}	100.0 {96.9}
C++	100.0 {100.0}	100.0 {100.0}

str.

Source \ Destination	Python	C++
Python	100.0 {96.9}	100.0 {93.8}
C++	96.9 {90.6}	100.0 {90.6}

list[str].

Source \ Destination	Python	C++
Python	100.0 {100.0}	100.0 {87.5}
C++	100.0 {93.8}	100.0 {96.9}

Table 8: Datatype breakdown of Table 1: causal success {source readout}, in percent. Each cell has 32 examples, using 250 heads, assignment pivots, task-span padding, OV amplification 2, and no collateral ablation.

G.2 Query transfer under a task-order contrast

Let A count characters, B count uppercase letters, C test whether any word is a palindrome, and D count vowel-initial words. The source orders (A, B, C, D) and has completed (A, B); the destination orders (A, B, D, C) and has completed only A. Its native next task is therefore B. A transferred state encoding only completed tasks (A, B) would make D next under destination order, whereas the source's next goal is C. The slide example shows a query-patched continuation implementing C, accompanied by a palindrome readout. This is a qualitative contrast, not a factorial evaluation of progress and goal identity.

```
...
(1) total num of characters
(2) total num of uppercase letters
(3) whether any word is a palindrome
(4) the number of words starting with a vowel
...
<code>
def func(words: list[str]):
    a = sum(len(w) for w in words)
    b = sum(1 for w in words for c in w if c.isupper())
...
```

```
...
(1) total num of characters
(2) total num of uppercase letters
(3) the number of words starting with a vowel
(4) whether any word is a palindrome
...
<code>
def func(words: list[str]):
    a = sum(len(w) for w in words)
...
```

Source (upper prompt): completed A, B; next C.

Destination (lower prompt): completed A; native next B.

Query patch: transfer the source pivot queries to destination goal heads.

Completed-task prediction: D.
Observed patched goal: C.

```
b = any(w == w[::-1] for w in words)
```

```
↳ goalscope · q-transfer  pos=141 '    ' => I'm checking if any element in the list satisfies the condition of being a palindrome.
```

Figure 12: **A query-transfer contrast from the exploratory slides.** The source and destination contain the same four tasks in different orders and have unequal prefills. The patched destination computes whether any word is a palindrome, matching the source's upcoming goal. Ellipses in the prompt panels mark the capture/patch boundary; the saved readout identifies position 141. This example does not establish that query states contain only goal information.

H OVREADER

For a residual state after source layer L, OVReader applies each selected head's value map and output map, including its pre-attention normalization gain. It unit-normalizes each transformed vector and averages them. This operator is based on model weights, but is not a single linear projection because of the per-head normalization. The aggregate steers a separate GoalScope probe with norm-relative coefficient .15; the selected-head-output comparison uses .1.

The evaluation samples 64 examples per datatype, with five capture positions and layer sweeps through the final layer. At the assignment operator, readout can approach the head-output reference around L40; pooled curves generally peak near L48–52. A decline at the final layer indicates reduced accessibility through this particular transformation and probe, not absence of all subgoal information. Implementation-transfer scores were rejudged with a trivial-output sanitizer; existing records should not be described as a freshly resampled corpus excluding all trivial target predicates.

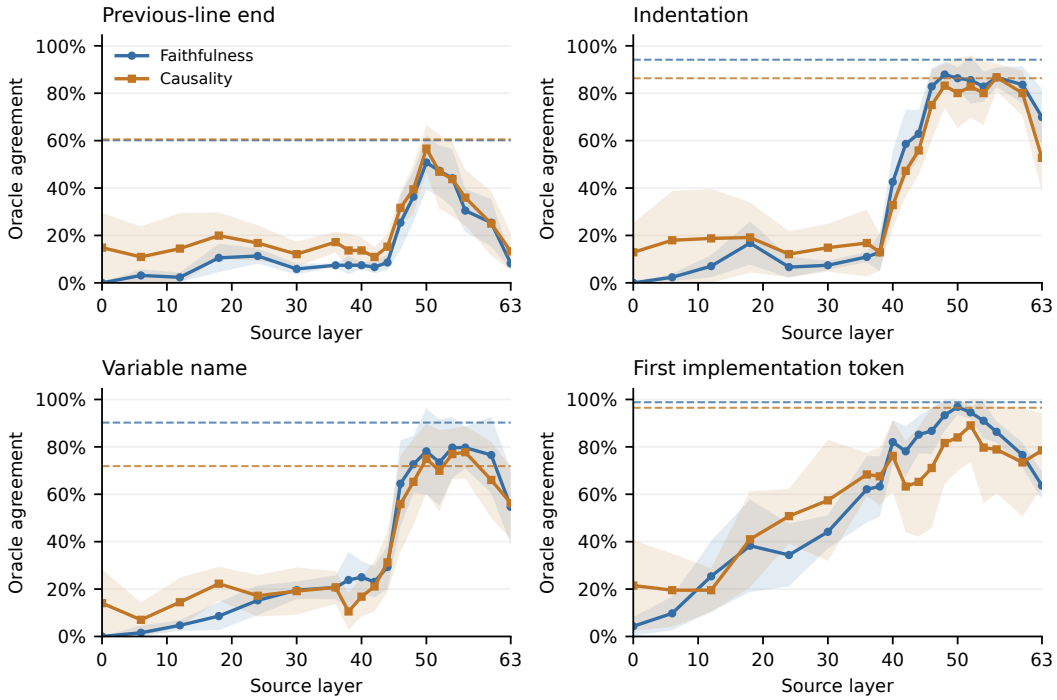


Figure 13: **OVReader at the remaining capture positions.** The cohort and aggregation match Figure 5: means and ± 1 sample standard deviation across four datatype rates, with 64 examples each. Dashed horizontal lines show the actual head-output reference for the same position. The first implementation token is included as a comparison after implementation begins.

The plots use the complete archived evaluation with rule-based frontier spans, rather than the incomplete frontier-span rerun. They contain 19 source-layer settings and a head-output reference, with 64 examples per datatype at every position and in both evaluation modes. The datatype bands describe variation across four task families; they are not confidence intervals over independent runs.

I ATTENTION TO TASK DESCRIPTIONS

The query-transfer example in Figure 12 also provides a qualitative view of attention to the destination task list. Its native frontier is the uppercase-count task; after transferring source queries, the continuation and readout instead pursue the palindrome predicate. The slide highlights both descriptions in the task list. This visualization illustrates the association between the heads' access to task tokens and the operation expressed by the continuation.

```
(1) whether any word is empty
(2) the number of words longer than five
characters
(3) the number of words that start with a vowel

<code>
def func(words: list):
    a = any(len(w) == 0 for w in words)
    b = sum(1 for w in words if len(w) > 5)
    c = sum(1 for w in words if
w.lower().startswith(('a', 'e', 'i', 'o', 'u')))
    return a, b, c
</code>
```

layer	OV(h)	write-proj	read-proj	eigen
36	NO I'm counting occurrences of a specific character in a string.	NO_ I'm processing the line of code to determine its functionality.	NO_ I'm not sure what specific line you're referring to, but it would be the count of …	NO_ I am processing the line to identify specific operations or operations involving u…
40	YES I'm checking if the character is a vowel.	NO_ I'm checking if a specific condition is met in the code.	YES I'm checking if the current character is a vowel (assuming the line is `sum(1 for …	NO This line checks whether the variable is a boolean (checking if it is a boolean o…
44	YES I'm checking if the variable starts with a vowel (a, e, i, o, u) using a simple co…	NO_ I'm checking if the variable `x` is being used to store values that match a specif…	YES I'm assuming you're referring to a line of code where a variable is initialized wi…	NO_ I am processing the line checking for specific patterns in the code, identifying w…
48	YES I'm checking for occurrences of characters in the string that match 'a', 'e', 'i',…	YES I'm checking for occurrences of specific characters in the string, counting how ma…	YES Sure, please provide the line of code and I'll count how many times it matches cer…	MEA I'm counting how many times the variable has been incremented, using a loop for ea…
52	YES Sure, I need to know which specific characters or patterns to look for in your tex…	YES I'm counting the number of vowels in a string, specifically for the vowels 'a', 'e…	YES Sure, I can help with that. Please provide the specific line of code you want me t…	YES I'm counting the number of occurrences of specific characters in the string, consi…
56	NO Sure, please specify the exact characters or keywords that should be considered fo…	NO_ I'm processing the line of code that involves checking for specific conditions rel…	YES Sure, I'll count the number of lines that start with a specific letter in your cod…	YES I'm assuming you're referring to a specific line of code, but you didn't specify w…

Figure 14: **Qualitative projection readouts from the exploratory slides.** The source has counted empty and long words; at the highlighted assignment it is about to count words beginning with a vowel. The slide compares an OV-transformed state with alternative write/read and eigenvector-based directions. These individual descriptions illustrate the experiment and are not additional aggregate evidence for the layer sweep.

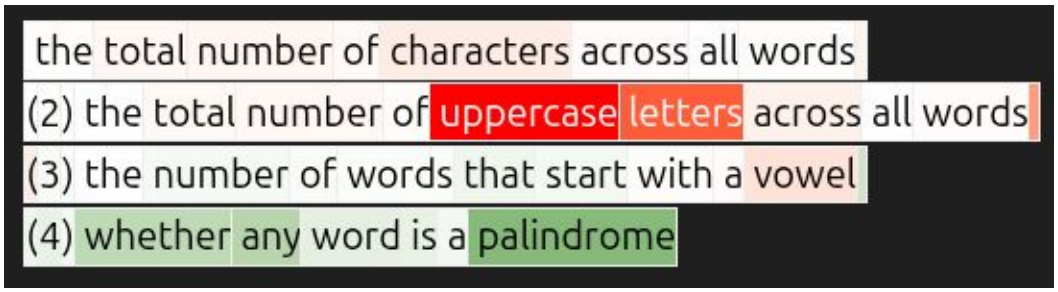


Figure 15: Task-description highlights accompanying the query-order experiment. The destination's native uppercase-count frontier is highlighted in red and the transferred palindrome goal in green. The slide supplies no calibrated color scale; this panel should be read as a qualitative visualization rather than a quantitative attention-mass comparison.

A quantitative account requires a specified attention statistic, token-span definition, and aggregation across heads and examples. We leave comparison with independently localized gaze or concept-copying heads to a separate evaluation.

J Generalization Across Model Families and Sizes

We repeat localization and evaluation within five models: Qwen2.5-Coder at 14B and 32B parameters, Qwen2.5-32B-Instruct, and OLMo-3.1-32B instruction and SFT variants. Each model uses its own selected heads and its own probe generation. This tests replication of readable and transferable task information, not activation transplantation between models.

J.1 Matched evaluation and head selection

All ten runs use the same 253 specification identifiers and five capture positions, yielding 1,265 records per model and mode. Rates below are recomputed from saved records. The intervention is head replacement with source coefficient 1 and steering throughout decoding; the probe generation budget is 64 tokens. Replacement operates at the selected heads' own layers and ignores the additive steer-band setting. Thus the results are not directly interchangeable with the main norm-relative additive scores.

Model	Heads	Selected layer range	Faithfulness	Transfer
Qwen Coder 32B	122	28–55	0.823	0.783
Qwen Coder 14B	88	18–47	0.598	0.401
Qwen Instruct 32B	108	29–55	0.308	0.540
OLMo Instruct 32B (+RL)	93	11–35	0.495	0.689
OLMo Instruct 32B (SFT)	91	11–34	0.595	0.533

Table 9: Within-model replication on the locked corpus, pooled over capture positions. Head sets are model-specific. A selected layer range gives its endpoints, not a claim that every layer is represented.

The Qwen sets contain 122, 88, and 108 heads, respectively. The OLMo comparison uses 93 instruction-model heads and 91 SFT-model heads from the candidate band 10–36, using the exported threshold selections rather than the older 130/79-head pair. Their counts are approximately matched, not identical. Head membership and its layer distribution remain model-specific. Full model identifiers and selection names are listed below for reproducibility.

`Qwen/Qwen2.5-Coder-32B-Instruct`
Selection: `goal`.

`Qwen/Qwen2.5-Coder-14B-Instruct`
Selection: `goal`.

`Qwen/Qwen2.5-32B-Instruct`
Selection: `goal`.

`allenai/Olmo-3.1-32B-Instruct`
Selection: `json:goal__lam20_thr1.0_band10-36`.

`allenai/Olmo-3.1-32B-Instruct-SFT`
Selection: `json:goal__lam20_thr0.6_band10-36`.

J.2 Results by capture position

Model	Metric	Prev. end	Indent	Variable	Assignment	First impl.
Qwen Coder 32B	Faith.	0.439	0.885	0.885	0.945	0.960
Qwen Coder 32B	Transfer	0.300	0.858	0.838	0.945	0.972
Qwen Coder 14B	Faith.	0.154	0.474	0.557	0.889	0.917
Qwen Coder 14B	Transfer	0.028	0.198	0.206	0.791	0.783
Qwen Instruct 32B	Faith.	0.000	0.245	0.071	0.636	0.589
Qwen Instruct 32B	Transfer	0.040	0.601	0.217	0.905	0.937
OLMo Instruct 32B (+RL)	Faith.	0.020	0.648	0.257	0.814	0.735
OLMo Instruct 32B (+RL)	Transfer	0.158	0.881	0.526	0.921	0.957
OLMo Instruct 32B (SFT)	Faith.	0.055	0.668	0.466	0.925	0.862
OLMo Instruct 32B (SFT)	Transfer	0.036	0.628	0.324	0.818	0.858

Table 10: Exact per-position rates for the model comparison in Figure 16.

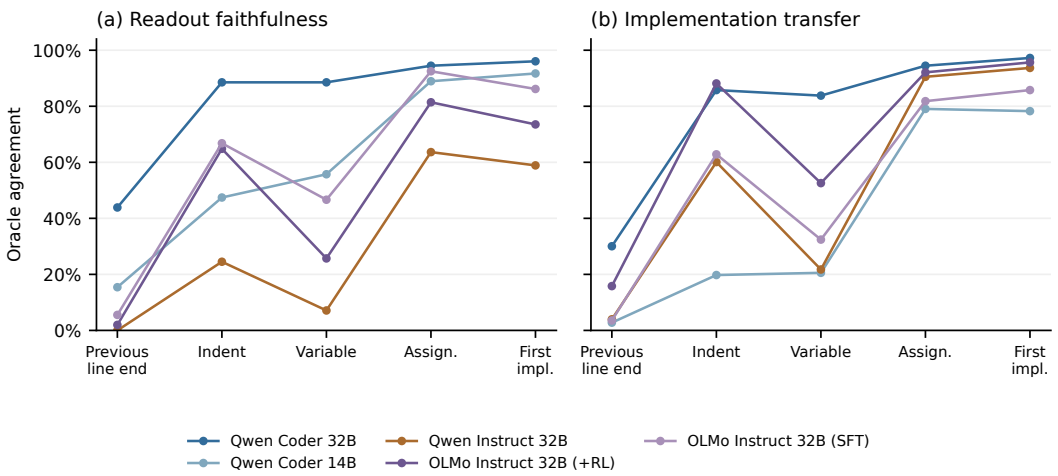


Figure 16: Readout faithfulness and implementation transfer within each model across capture positions (253 records per point). The first implementation token is separated from the four earlier positions. The common scale exposes differences in timing and metric, which pooled rates can obscure.

J.3 Interpretation

The Coder-32B configuration scores above Coder-14B on both pooled metrics; it also outperforms the generic Qwen-32B instruction configuration on these code tasks. For OLMo, the instruction model has higher implementation-transfer scores, while the SFT model has higher readout faithfulness. Replication therefore does not imply identical performance or a single ordering of models across metrics.

These comparisons vary checkpoints, head sets, and potentially source-generation competence together. They support the presence of the phenomenon across families and sizes, but do not isolate causal effects of scale, code training, or RL. Capturing at equivalent named boundaries also need not mean identical tokenization across models. The approximate OLMo budget matching reduces one disparity without controlling every difference. All scores use the partial-content oracle rubric in Appendix D, rather than execution-based program correctness.

K READOUTS WITHOUT ENUMERATED SUB-TASKS

The controlled evaluation names the component predicates in the instruction. The examples below instead read intermediate operations from familiar programs whose component steps are not separately enumerated. They are qualitative illustrations from the exploratory slides, not part of the 253-specification evaluation.

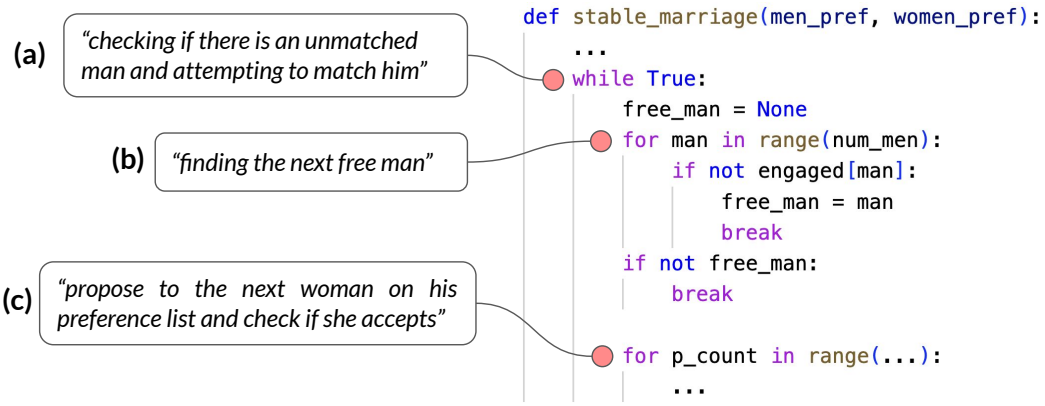


Figure 17: **Reading intent while writing code.** At the marked positions, GoalScope reads descriptions of the operations the model is about to implement: (a) checking for an unmatched man, (b) finding the next free man, and (c) proposing to the next woman on his preference list. These descriptions express an operation rather than a particular token sequence; for example, finding an unmatched man can be implemented with different loop forms. The figure illustrates this distinction and does not itself compare readouts across alternative implementations. Later code is shown for context.

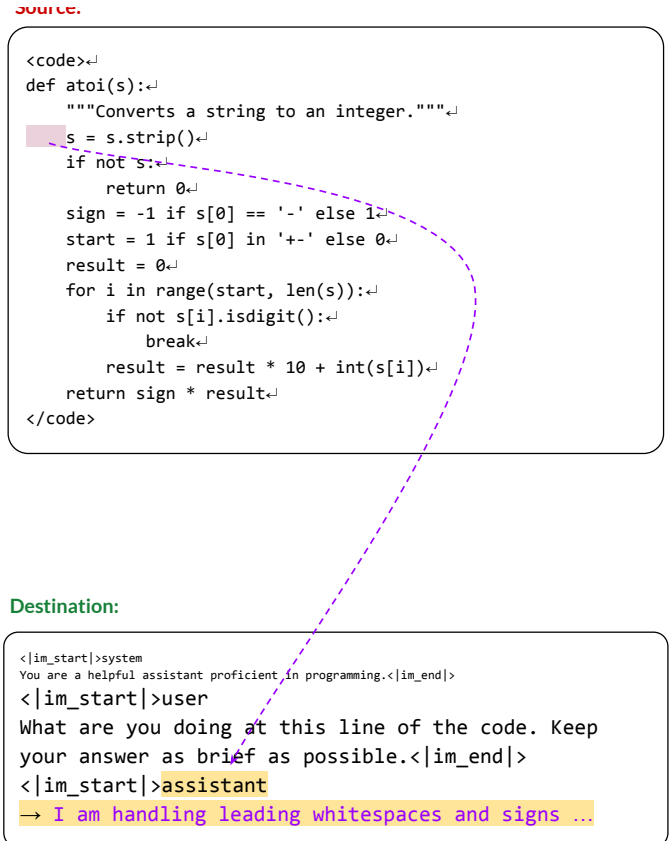


Figure 18: **A free-form string-to-integer example.** The highlighted source position is the indentation before `s = s.strip()`. A separate generic probe produces the displayed description about leading whitespace and signs. The later program is shown for context, but a cache at the highlighted token cannot depend on that future text. The description is a qualitative readout and need not match the exact granularity of the immediately following statement.

L GENERALIZATION TO NON-CODE SETTINGS

L.1 FEW-SHOT ANTONYM EXAMPLE

The saved exploratory run in `19_goalscope_out_of_domain.ipynb` uses six demonstrations: enter/exit, feasible/infeasible, manned/unmanned, fuzzy/smooth, arise/settle, and far/near. The prompt ends at `Q: mercy` followed by `A:`. Its continuation starts with "cruelty" and then generates further question–answer pairs. The code-localized 122-head set supplies norm-relative additive readouts with $\alpha = .1$, injection layers 35–45, and repeated steering during readout generation.

The probe uses the system instruction "You are a helpful assistant." and the user instruction "You are answering a series of questions that all follow the same hidden rule. In one short phrase, what task are you performing?" This is a domain-adapted probe, not the original code-oriented prompt. Saved token indices refer to the notebook's tokenized prompt plus continuation.

Position	Source token	Readout
57	newline after cruelty	Identifying antonyms.
60	light	Identifying patterns.
70	A before slow	Identifying antonyms.
71	colon before slow	Decoding or interpreting slower actions.

Table 11: Selected qualitative readouts from one saved antonym run, including a vague description and a mismatch. These are not a quantitative evaluation sample.

L.2 OTHER EXPLORATORY DOMAINS

Recipe, travel, and narrative prompts use adapted verbalization questions. These settings require separate decisions about the granularity and correctness of the next step; they are not included in the code results or claimed as a validated general-purpose planning readout. A systematic evaluation remains to be completed.

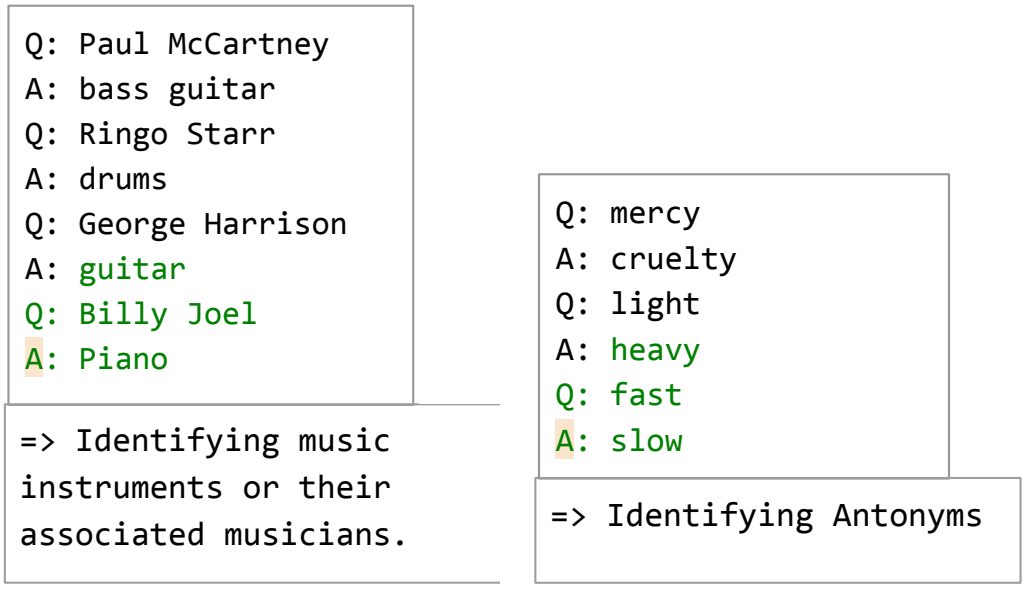


Figure 19: **Reading a relation from few-shot examples.** The slide shows musician–instrument and antonym prompts, with capture positions highlighted on the answer prefix `A`. GoalScope describes the inferred relations. Green text denotes generated continuation shown for context. These selected examples are exploratory and are distinct from the token-indexed antonym run above.

M BOUNDARY SIGNALS: MOVING ON AND STAYING

This supporting line of experiments asks whether signals associated with ending a subgoal can influence when the model advances, independently of the semantic identity read by GoalScope. Prompts permit a predicate to occupy one or several lines, with repeated lines beginning with the current predicate's variable. At a line boundary, the next variable therefore indicates whether the model stays with that predicate or moves to the next one.

Source/destination contrasts are constructed so the source's next variable, the destination's native next variable, and the desired counterfactual variable differ. Head-output patching is optimized to promote the counterfactual continuation, reducing direct copying of a source variable as an explanation. Transfers from completed to incomplete steps can induce moving on; reverse transfers can induce staying or repetition. Variable renaming, cross-language examples, and reversed instruction order probe the reach of this signal.

These observations motivate a distinction between subgoal identity and transitions between subgoals. A next-variable change alone does not show correct implementation of the next predicate, and repeated output under a stay intervention requires separate interpretation. Full continuation scoring and tests of changes to goal readouts are needed before claiming a complete causal dissociation. Boundary experiments are therefore supporting material rather than a premise of the main subgoal-representation account.

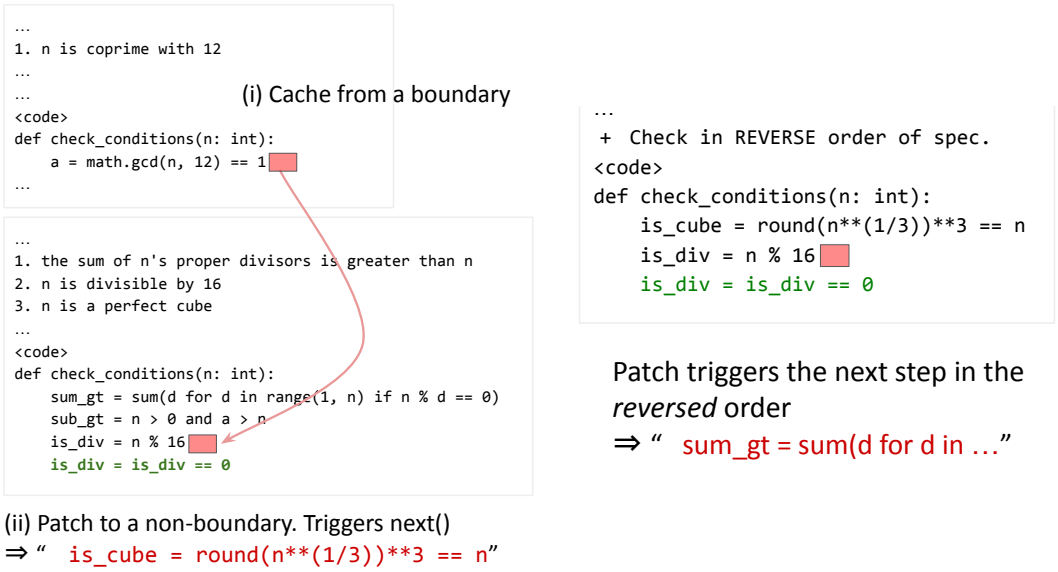


Figure 20: **A boundary intervention under different task orders.** Left: capture at the highlighted completed source expression and patch at the highlighted unfinished divisibility expression in the destination. The green continuation is the native completion of that expression; red text shows the patched run advancing to a new variable. Right: under reversed instruction order, the patched continuation advances to the next task in that order. These are selected examples; advancing to a variable does not certify that the preceding or following predicate is implemented correctly.

N GENERALIZATION ACROSS PROGRAMMING STYLES AND LANGUAGES

O CHECKING FOR FUNCTIONAL DISSOCIATION

O.1 GOAL HEADS VS IMPULSE HEADS

P ROLE OF MLPS

arnab: TODO: path patching experiments: patch + freeze MLPs with dest values. Measure intervention effects.